

# ソーラーカー MPC プログラムを読むための基礎知識

solar\_ws0129-main

2026-07-29

## Contents

|    |                                               |    |
|----|-----------------------------------------------|----|
| 1  | プログラム、プロセス、メモリ、オブジェクトを区別する                    | 3  |
| 2  | 名前、代入、型、参照                                    | 3  |
| 3  | 関数、引数、戻り値、スコープ                                | 3  |
| 4  | class、独自クラス、インスタンス、self、継承                    | 4  |
| 5  | 先頭アンダースコア、__init__、名前付けの作法                    | 4  |
| 6  | デコレータと @dataclass                             | 4  |
| 7  | module、package、import、console_scripts         | 5  |
| 8  | 例外、try/finally、with、資源解放                      | 5  |
| 9  | list、dict、deque、NumPy 配列、pandas DataFrame     | 6  |
| 10 | CLI、PowerShell、Bash、環境変数、終了コード                | 6  |
| 11 | rclpy、rcl、rmw、DDS/RTPS の層                     | 6  |
| 12 | ROS graph、Node、topic、service、action、parameter | 6  |
| 13 | callback、timer、Executor、spin、Callback Group   | 7  |
| 14 | launch Action、Node Action、実行可能名、remapping     | 7  |
| 15 | rqt_graph、ros2 CLI、rosviz をいつ使うか              | 7  |
| 16 | 制御、状態、入力、モデル予測制御 MPC                          | 8  |
| 17 | 上位 MPC と下位 MPC の役割                            | 8  |
| 18 | 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明              | 9  |
| 19 | Cross-Entropy Method を式と実装で理解する               | 9  |
| 20 | warm start は何を保存し、どう効くか                       | 10 |

|                                              |    |
|----------------------------------------------|----|
| 21 車両力学、電力収支、効率 map                          | 10 |
| 22 SoC、内部抵抗、端子電圧、温度、MHE                      | 10 |
| 23 天候、route、補間、時刻、単位                         | 11 |
| 24 freshness、filter、guard、fallback、fail-safe | 11 |
| 25 CSV/YAML の data contract と検証              | 11 |
| 26 単体試験、SILS、replay、観測可能性                    | 11 |

本総論は、各プログラム解説で前提になる Python、OS、ROS 2、数値最適化、MPC、車両・電池モデルを、初学者向けに説明する。各個別 PDF にも、そのファイルに必要な章を再掲する。

## 1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘\*args’、‘\*\*kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 4 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、`rcpy` の `Node` を基底クラスとして継承し、`Node` が持つ `publisher`、`subscription`、`timer`、`parameter` などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 `self` として ‘\_\_init\_\_’ へ渡す。‘self’ は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().\_\_init\_\_(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create\_publisher’などを正しく使えない。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

## 5 先頭アンダースコア、\_\_init\_\_、名前付けの作法

‘self\_step\_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘\_\_init\_\_’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘\_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘\_base\_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6 デコレータと @dataclass

‘@名前’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘@Class’という一般構文があるのではなく、‘@dataclass’など実際のデコレータ名を書く。

‘@dataclass’は型注釈付きフィールドから‘\_\_init\_\_’、‘\_\_repr\_\_’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は‘\_\_post\_init\_\_’や別の検証関数で確認する必要がある。

## 根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

## 7 module、package、import、console\_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console\_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

## 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

## 8 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

## 9 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 '(N)」、候補集団なら '(population, N) となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

## 10 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。'argparse' は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar\_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS\_DOMAIN\_ID、RMW\_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

### 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

## 11 rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscription などを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executor の実行モデルは rcl だけで完結せず client library 側にも実装される。

### 根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

## 12 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

## 根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

## 13 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self.on\_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は MultiThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る

## 根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

## 14 launch Action、Node Action、実行可能名、remapping

'launch\_ros.actions.Node(...)' は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

'DeclareLaunchArgument' は launch 実行時に受け取る入力欄を宣言し、'LaunchConfiguration' はその値を後で解決する substitution を表す。'perform(context)' は実行時 context から確定文字列を取り出す。

launch の 'name' は Node 名 override、'parameters' は起動時 parameter、'remappings' は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

## 根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

## 15 rqt\_graph、ros2 CLI、rosvbag2 をいつ使うか

rqt\_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。



```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

## 根拠資料

- [ROS 2 Humble 公式: rqt\\_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

## 16 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を  $x$ 、操作入力を  $u$ 、外乱・予報を  $w$  とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から  $N$  ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

## 根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

## 17 上位 MPC と下位 MPC の役割

上位層は長い距離または時間を見て、エネルギー、到着、停止、速度制限、終端 SoC を考えた速度計画を出す。下位層は短い周期で実測速度を見て、上位速度へ追従する駆動・回生入力を求める。



予報・route・SoC -> 上位MPC -> 将来速度列 -> 下位MPC -> throttle/回生/drive mode -> driverまたはvehicle -> 新しいtelemetry

上位解が遅い間も下位出力を止めないこと、古い計画を安全に保持すること、上位と下位で単位と時刻基準を一致させることが実装上重要である。

## 18 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

### 根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

## 19 Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった elite 候補から分布を更新する反復的な確率最適化である。このリポジトリの upper\_solver.py は各制御点速度を独立正規分布で生成し、上下限へ clip する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$

$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left( \text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで  $\mathcal{E}_g$  は cost が小さい上位  $K$  候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 hybrid\_bounded\_minimize は、deterministic seed を評価し、上位候補を L-BFGS-B で局所 refine し、設定と seed 間不一致に応じて CEM を実行し、最後に再度局所 refine する。したがって CEM 単独ではなく hybrid solver である。

CEM で落とした候補を永久保存しないこと自体は通常最適化として自然だが、off-nominal 状態からの再利用には別の policy library 設計が必要である。状態を無制限に全組合せ保存する代わりに、SoC、進捗、時刻、予報誤差、停止状態などの scenario を設計し、近傍 policy を検索して MPC で再最適化する。

### 根拠資料

- Rubinstein and Kroese: [The Cross-Entropy Method](#)
- SciPy 公式: [scipy.optimize.minimize](#)

## 20 warm start は何を保存し、どう効くか

warm start は前回または offline 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 library の一員を与える。

$$u_i^{0,\text{new}} = \text{interp} \left( s_i^{\text{new}}; s_j^{\text{old}}, u_j^{*,\text{old}} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して shift する。初期状態や天候が変われば cost 評価は新条件で行われるので、warm start が不適切でも他 seed、CEM、安全 fallback が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 mean または候補 pool、receding horizon なら前回解の時間・距離 shift として現れる。どの位置に渡しているかを呼出引数まで追う。

### 根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- Rubinstein and Kroese: [The Cross-Entropy Method](#)

## 21 車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 power は概ね ‘P\_mech = F\_total \* v + P\_inertia’ で、駆動時と回生時で異なる効率 map を通して DC 側 power へ変換する。map は速度・torque などを軸にした実測または同定 table であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

## 22 SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{\text{oc}}(z, T) - IR_{\text{total}}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T) I \Delta t}{Q_{\text{eff}}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

### 根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

## 23 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻  $t$  と距離  $s$  が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 `timezone`、`naive datetime` を混ぜると数時間のずれが発生する。入力で `timezone` を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

## 24 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と `timeout` から `freshness` を判定し、`stale` 値を計画状態へ無条件に同期しない。

`filter` は `noise` と一時的な飛び値を抑えるが、遅れを生む。`slew limit` は指令変化率を制限する。安全 `guard` は `solver` の `cost` 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

`fallback` は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 `mode` ごとに選ぶ。`fallback` 発生は `status` と `log` へ残し、正常解と区別する。

## 25 CSV/YAML の data contract と検証

`data contract` は列名、型、単位、`timezone`、欠損可否、並び順、重複、許容範囲、先頭行、`encoding` を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、`map`、`route`、`forecast`、`stop`、`schedule` はそれぞれ `grain` が異なる。生成時に `schema validation`、物理範囲、時間単調性、`route` 範囲、`coverage` を検査し、検査結果を `artifact` として残す。

学習用データと独立検証データを分離し、RMSE だけでなく `bias`、時系列残差、`energy` 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

## 26 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、`historical replay` は実測入力への再現性、本番 `preflight` は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、`solver` 成否、候補数、計算時間、`fallback`、`source revision`、`profile hash` を同じ `run ID` で記録する。

非同期不具合は最終値だけでは追えない。`topic` 周期、`message age`、`callback` 所要時間、`queue` 遅延、Node 生存、`publisher` 数を同時に観測する。

### 根拠資料

- [ROS 2 Humble 公式: rqt\\_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)